

Add scope_association concept to P3149

Document #: P3815R1
Date: 2025-11-07
Project: Programming Language C++
Audience: LEWG Library Evolution Working Group
Reply-to: Ian Petersen
<ispeters@gmail.com>
Jessica Wong
<jesswong2011@gmail.com>

Contents

- 1 Changes
 - 1.1 R1
 - 1.2 R0
- 2 Background and Motivation
 - 2.1 execution::spawn
 - 2.2 execution::associate
- 3 Proposal
 - 3.1 execution::scope_token
 - 3.2 execution::associate
 - 3.3 execution::spawn
 - 3.4 execution::spawn_future
 - 3.5 execution::simple_counting_scope
 - 3.6 execution::counting_scope
- 4 Wording
 - 4.1 Header <execution> synopsis 33.4 [execution.syn]
 - 4.2 execution::associate
 - 4.3 execution::spawn_future
 - 4.4 execution::spawn
 - 4.5 Async scope concepts
 - 4.6 execution::simple_counting_scope and execution::counting_scope
- 5 References

§ 1 Changes

§ 1.1 R1

- Incorporate feedback from Ville to strengthen background and motivation section.

§ 1.2 R0

§ 2 Background and Motivation

[P3149R11] was approved for C++26 by WG21. The paper introduces two scope types, `simple_counting_scope` and `counting_scope`, along with several basis operations, including `associate`, `spawn`, and `spawn_future`. In R11, the example implementations of these facilities are expressed in terms of the `scope_token` concept:

```
template <class Token>
concept scope_token =
    copyable<Token> &&
    requires(const Token token) {
        { token.try_associate() } -> same_as<bool>;
        { token.disassociate() } noexcept -> same_as<void>;
        { token.wrap(declval<test-sender>()) } -> sender_in<test-env>;
    };
```

In [P3149R7], we also introduced the `scope_association` concept as an RAII handle for managing scope associations. However, this facility was removed in [P3149R9] due to concerns about the unconventional behavior of its copy constructor. As there was no implementation experience with `scope_association` at the time of R9’s approval, the implications of this removal was not fully understood.

Later implementation efforts demonstrated that reintroducing the `scope_association` concept from [P3149R7] would yield several benefits for sender/receiver library implementors:

- reduced binary size for both scopes and basis operations – i.e., the use of a boolean is no longer needed to track successful associations in the op states of senders,
- fewer move/copy operations,
- and a simplified implementation model that in turn facilitates easier customization – i.e., library implementors do not need to manually track successful associations to correctly handle disassociate calls.

These improvements can be achieved without any impact on the user-facing APIs proposed in R11.

Illustrated below are the R11 implementations of `spawn` and `associate` in contrast with the `scope_association` concept implementation.

§ 2.1 execution::spawn

Before	After
<pre>template <class Alloc, scope_token Token, sender Sender> struct spawn-state : spawn-state-base { using op-t = connect_result_t<Sender, spawn- receiver>; spawn-state(Alloc a, Sender&& sndr, Token t) : alloc(std::move(a)), op(connect(std::move(sndr), spawn-receiver{this})), token(std::move(t)) {} void run() { if (token.try_associate()) op.start(); else destroy(); } }</pre>	<pre>template <class Alloc, scope_token Token, sender Sender> struct spawn-state : spawn-state-base { using op-t = connect_result_t<Sender, spawn- receiver>; using assoc-t = remove_cvref_t<decltype(declval<Token>> ().try_associate())>; spawn-state(Alloc a, Sender&& sndr, Token t) : alloc(std::move(a)), op(connect(std::move(sndr), spawn-receiver{this}))) { assoc = t.try_associate(); } void run() noexcept {</pre>

Before	After
<pre> void complete() noexcept override { auto t = std::move(token); destroy(); t.disassociate(); } }; </pre>	<pre> if (assoc) op.start(); else destroy(); } void complete() noexcept override { auto a = std::move(assoc); destroy(); } }; </pre>

In the above example, a key difference when implementing *spawn-state* with a *scope_association* concept is that `t.try_associate()` is invoked in the constructor rather than in `run()`. This change simplifies exception handling within *spawn-state*: if an exception occurs, only the allocation needs to be explicitly cleaned up. By contrast, if `t.try_associate()` throws from `run()`, both destruction and deallocation needs to be explicitly handled.

§ 2.2 execution::associate

Before	After
<pre> template <scope_token Token, sender Sender> struct associate-data { explicit associate-data(Token t, Sender&& s) noexcept(noexcept(t.wrap(std::forward<Sender>(s))) && noexcept(t.try_associate())) : token(std::move(t)), sndr(token.wrap(std::forward<Sender>(s))) { if (!token.try_associate()) { sndr.reset(); } } associate-data(const associate-data& other) noexcept(is_nothrow_copy_constructible_v<wrap_sender> && noexcept(other.token.try_associate())) requires copy_constructible<wrap_sender> : token(other.token) { if (other.sndr.has_value() && token.try_associate()) { try { sndr.emplace(*other.sndr); } catch (...) { token.disassociate(); throw; } } } associate-data(associate-data&& other) noexcept(is_nothrow_move_constructible_v<wrap_sender>) : sndr(std::move(other).sndr), token(std::move(other).token) { other.sndr.reset(); } </pre>	<pre> template <scope_token Token, sender Sender> struct associate-data { explicit associate-data(Token t, Sender&& s) noexcept(noexcept(t.wrap(std::forward<Sender>(s))) && noexcept(t.try_associate())) : sndr(t.wrap(std::forward<Sender>(s))) { sender_ref guard{addressof(sndr)}; assoc = t.try_associate(); if (assoc) { (void)guard.release(); } } associate-data(const associate-data& other) noexcept(is_nothrow_copy_constructible_v<wrap_sender> && is_nothrow_copy_constructible_v<assoc_t>) requires copy_constructible<wrap_sender> : assoc(other.assoc) { if (assoc) { construct_at(addressof(sndr), other.sndr); } } associate-data(associate-data&& other) noexcept(is_nothrow_move_constructible_v<wrap_sender>) : associate- data(std::move(other).release()) {} ~associate-data() { if (assoc) { destroy_at(addressof(sndr)); } } </pre>

Before	After
<pre> ~associate-data() { sndr.reset(); } optional<pair<Token, wrap_sender>> release() && noexcept(is_nothrow_move_constructible_v<wrap- sender>) { if (sndr) { return optional{ pair{std::move(token), std::move(*sndr)}}; } else { return nullptr; } } private: optional<wrap_sender> sndr; Token token; }; </pre>	<pre> pair<assoc_t, sender_ref> release() && noexcept { wrap_sender* p = assoc ? addressof(sndr) : nullptr; return pair{std::move(assoc), sender_ref{p}}; } private: assoc_t assoc; union { wrap_sender sndr; }; associate-data(pair<assoc_t, sender_ref> parts) : assoc(std::move(parts.first)) { if (assoc) { construct_at(addressof(sndr), std::move(parts.second)); } } }; </pre>

§ 3 Proposal

```

template <class Assoc>
concept scope_association =
    movable<Assoc> &&
    default_initializable<Assoc> &&
    requires(Assoc assoc) {
        { static_cast<bool>(assoc) } noexcept;
        { assoc.try_associate() } -> same_as<Assoc>;
    };

```

A type that models `scope_association` is an RAII handle that represents a possible association between a sender and an async scope. If the scope association contextually converts to true then the object is “engaged” and represents an association; otherwise, the object is “disengaged” and represents the lack of an association. Scope associations are movable and not copyable, and expose a `try_associate` member function with semantics identical to the `try_associate` member function on a type that models `scope_token`.

The following are the proposed changes to `scope_token`, `associate`, `spawn`, `spawn_future`, `simple_counting_scope`, and `counting_scope` with the adoption of `scope_association`.

§ 3.1 execution::scope_token

The primary change to `scope_token` is to `try_associate`, which will return a `scope_association` rather than a `bool`.

```

template <class Token>
concept scope_token =
    copyable<Token> &&
    requires(Token token) {
        { token.try_associate() } -> scope_association;
        { token.wrap(declval<test_sender>()) } -> sender_in<test-env>;
    };

```

The `try_associate` member function on a token attempts to create a new association with the scope; `try_associate` returns an engaged association when the association is successful, and it may either return a disengaged association or throw an exception to indicate failure.

§ 3.2 `execution::associate`

With the application of the proposed changes, the copy behavior of the `associate-sender` returned from `associate` becomes the following:

If the sender, `snd`, provided to `associate()` is copyable then the resulting `associate-sender` is also copyable, with the following rules:

- copying an unassociated `associate-sender` invariably produces a new unassociated `associate-sender`; and
- copying an associated `associate-sender` requires copying the *associate-data* it contains and the *associate-data* copy-constructor proceeds as follows:
 - The result of invoking the source's `association.try_associate()` will be passed to the destination *associate-data*.
 - if the resulting association is engaged then copy the wrapped sender from the source into the destination *associate-data*; the destination `associate-sender` is associated
 - otherwise, the destination `associate-sender` is unassociated

Furthermore, the *operation-state*'s destructor becomes the following:

An *operation-state* with its own association must invoke the association's destructor as the last step of the *operation-state*'s destructor.

§ 3.3 `execution::spawn`

The behavior of `spawn` remains largely unchanged, with the primary difference being that *op_t* now holds an *association* rather than a *token*. Upon completion of the *operation-state*, the destructor of the *association* is invoked, replacing the previous mechanism of explicitly calling `token.disassociate()` on the local copy of the *token*.

§ 3.4 `execution::spawn_future`

The changes to `spawn_future` reflect the same changes proposed in `spawn`.

§ 3.5 `execution::simple_counting_scope`

The behavior of `simple_counting_scope` remains largely unchanged, with the primary difference being that the disassociation is handled by the destructor of the association returned from `token.try_associate()`.

§ 3.6 `execution::counting_scope`

The changes to `counting_scope` reflect the same changes proposed in `simple_counting_scope`.

§ 4 Wording

§ 4.1 Header `<execution>` synopsis 33.4 [[execution.syn](#)]

To the `<execution>` synopsis 33.4 [[execution.syn](#)], make the following change:

```
// [exec.scope]
// [exec.scope.concepts], scope concepts
template <class Token>
    concept scope_association = see below;
```

```
template <class Token>
    concept scope_token = see below;
```

§ 4.2 execution::associate

To the subsection 33.9.12.16 [exec.associate], make the following changes:

- Let `associate-data` be the following exposition-only class template:

```
namespace std::execution {

template <scope_token Token, sender Sender>
struct associate-data {                                // exposition only
    using wrap_sender =                               // exposition only
        remove_cvref_t<decltype(declval<Token>().wrap(declval<Sender>()))>;
    using assoc_t =                                     // exposition only
        decltype(declval<Token>().try_associate());

    using sender_ref =                                   // exposition only
        unique_ptr<wrap_sender, decltype([](auto* p) noexcept { destroy_at(p); })>;

    explicit associate-data(Token t, Sender&& s)
    : sndr(t.wrap(std::forward<Sender>(s))),
      token(t) {
      assoc([&] {
          sender_ref guard{addressof(sndr)};
          auto assoc = t.try_associate();
          if (assoc)
              guard.release();
          return assoc;
          })();

      if (!token.try_associate())
          sndr.reset();
    }

    associate-data(const associate-data& other)
    noexcept(is_nothrow_copy_constructible_v<wrap_sender> &&
             noexcept(other.tokenassoc.try_associate()));

    associate-data(associate-data&& other)
    noexcept(is_nothrow_move_constructible_v<wrap_sender>);
    : associate-data(std::move(other).release()) {}

    ~associate-data();

    optional<pair<Token, wrap_sender>>
    pair<assoc_t, sender_ref>
    release() && noexcept(is_nothrow_move_constructible_v<wrap_sender>);

private:
    optional<wrap_sender> sndr; // exposition only
    Token token; // exposition only

    associate-data(pair<assoc_t, sender_ref> parts); // exposition only

    union {
        wrap_sender sndr; // exposition only
    };
    assoc_t assoc; // exposition only
};

template <scope_token Token, sender Sender>
    associate-data(Token, Sender&&) -> associate-data<Token, Sender>;

}
```

- 3 For an *associate-data* object *a*, ~~*a.sndr.has_value()*~~ is *bool(a.assoc)* is true if and only if an association was successfully made and is owned by *a*.

```
associate-data(const associate-data& other)
    noexcept(is_nothrow_copy_constructible_v<wrap-sender> &&
              noexcept(other.tokenassoc.try_associate()));
```

- 4 Constraints: *wrap-sender models copy_constructible<wrap-sender>* is true.
- 5 Effects: ~~Value-initializes *sndr* and initializes *token* with *other.token*. If *other.sndr.has_value()* is false, no further effects; otherwise, calls *token.try_associate()* and, if that returns true, calls *sndr.emplace(*other.sndr)* and, if that exits with an exception, calls *token.disassociate()* before propagating the exception. Initializes *assoc* with *other.assoc.try_associate()*. If *bool(assoc)* is true initializes *sndr* with *other.sndr*.~~

```
associate-data(associate-data&& other)
    noexcept(is_nothrow_move_constructible_v<wrap-sender>);
```

- 6 Effects: ~~Initializes *sndr* with *std::move(other.sndr)* and initializes *token* with *std::move(other.token)* and then calls *other.sndr.reset()*.~~

```
associate-data(pair<assoc-t, sender-ref> parts);
```

- 6 Effects: Initializes *assoc* with *std::move(parts.first)*. If *bool(assoc)* is true initializes *sndr* with *std::move(*parts.second)*.

```
~associate-data();
```

- 7 Effects: ~~If *sndr.has_value()* returns false then no effect; otherwise, invokes *sndr.reset()* before invoking *token.disassociate()*. If *bool(assoc)* is true destroys *sndr*.~~

```
optional<pair<Token, wrap-sender>>
pair<assoc-t, sender-ref>
    release() && noexcept(is_nothrow_move_constructible_v<wrap-sender>);
```

- 8 Effects: ~~If *sndr.has_value()* returns false then returns an optional that does not contain a value; otherwise returns an optional containing a value of type *pair<Token, wrap-sender>* as if by:~~

```
return optional(pair(token, std::move(*sndr)));
```

Constructs an object *u* of type *sender-ref* that is initialized with *addressof(sndr)* if *bool(assoc)* is true and with *nullptr* otherwise, then returns *pair{std::move(assoc), std::move(u)}*.

- 9 Postconditions: ~~*sndr* does not contain a value.~~

- 10 The name *associate* denotes a pipeable sender adaptor object. For subexpressions *sndr* and *token*, if *decltype((sndr))* does not satisfy *sender*, or *remove_cvref_t<decltype((token))>* does not satisfy *scope_token*, then *associate(sndr, token)* is ill-formed.

...

- 13 The member *impls-for<associate_t>::get-state* is initialized with a callable object equivalent to the following lambda:

```
[<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(see below) {
    auto&& [_ , data] = std::forward<Sndr>(sndr);

    auto dataParts = std::move(data).release();

    using scope_token = decltype(dataParts->first);
    using wrap_sender = decltype(dataParts->second);
```

```

using associate_data t = remove_cvref_t<decltype(data)>;
using assoc_t = associate_data t::assoc_t;
using sender_ref t = associate_data t::sender_ref;

using op_t = connect_result_t<wrap_sendertypename sender_ref t::element_type, Rcvr>;

struct op_state {
    boolassoc_t associated = false;    // exposition only
    union {
        Rcvr* rcvr;                    // exposition only
        struct {
            scope_token token;      // exposition only
            op_t op;                // exposition only
            assoc;                  // exposition only
        };
    };

    explicit op_state(Rcvr& r) noexcept
    : rcvr(addressof(r)) {}

    explicit op_state(scope_token_tkn, wrap_sender&& sndr, Rcvr& r) try
    : associated(true),
    assoc(tkn, connect(std::move(sndr), std::move(r))) {}
    }
    catch (...) {}
    tkn.disassociate();
    throw;
    }

    explicit op_state(pair<assoc_t, sender_ref t> parts, Rcvr& r)
    : assoc(std::move(parts.first)) {}
    if (assoc)
    : new (voidify(op)) op_t(
    connect(std::move(*parts.second), std::move(r)));
    else
    rcvr = addressof(r);
    }

    explicit op_state(associate_data t&& ad, Rcvr& r)
    : op_state(std::move(ad).release(), r) {}

    explicit op_state(const associate_data t& ad, Rcvr& r)
    requires copy_constructible<associate_data t>
    : op_state(associate_data t(ad).release(), r) {}

    op_state(op_state&&) = delete;

    ~op_state() {
        if (associated) {}
        assoc.op.~op_t();
        assoc._token_.disassociate();
        assoc._token_.~scope_token();
    }
    }

    void run() noexcept {    // exposition only
        if (associated)
        start(assoc.op);
        else
        set_stopped(std::move(*rcvr));
    }
};

if (dataParts)
return op_state{std::move(dataParts->first), std::move(dataParts->second), rcvr};
else

```



```

    return op_state{[std::forward_like<Sndr>(data),\ ]{.add}@rcvr};
}

```

- 14 The expression in the noexcept clause of `impls-for<associate_t>::get-state` is

```

is_nothrow_constructible_v<remove_cvref_t, Sndr> &&
is_nothrow_move_constructible_v<wrap_sender> &&
(is_same_v<Sndr, remove_cvref_t> ||
 is_nothrow_constructible_v<remove_cvref_t<Sndr>, Sndr>) &&
nothrow_callable<connect_t, wrap_sender, Rcvr>

```

where `wrap_sender` is the type `remove_cvref_t<data-type<Sndr>>::wrap_sender`.

§ 4.3 execution::spawn_future

To the subsection 33.9.12.18 [\[exec.spawn.future\]](#), make the following changes:

- 7 Let `spawn-future-state` be the exposition-only class template:

```

namespace std::execution {
    template <class Alloc, scope_token Token, sender Sender, class Env>
    struct spawn-future-state // exposition
    {
        only
        : spawn-future-state-base<completion_signatures_of_t<future-spawned-sender<Sender, Env>>>
        {
            using sigs-t = // exposition
            only
            completion_signatures_of_t<future-spawned-sender<Sender, Env>>;
            using receiver-t = // exposition
            only
            spawn-future-receiver<sigs-t>;
            using op-t = // exposition
            only
            connect_result_t<future-spawned-sender<Sender, Env>, receiver-t>;

            spawn-future-state(Alloc alloc, Sender&& sndr, Token token, Env env) // exposition
            only
            : alloc(std::move(alloc)),
              op(connect(
                  write_env(stop-when(std::forward<Sender>(sndr), source.get_token()),
                    std::move(env)),
                  receiver-t(this)),
                token(std::move(token)),
                associated(token.try_associate()) {
                    if (associatedassoc)
                        start(op);
                    else
                        set_stopped(receiver-t(this));
                }

            void complete() noexcept override; // exposition
            only
            void consume(receiver auto& rcvr) noexcept; // exposition
            only
            void abandon() noexcept; // exposition
            only

        private:
            using alloc-t = // exposition
            only
            typename allocator_traits<Alloc>::template rebind_alloc<spawn-future-state>;
            using assoc-t =
            // exposition only
            remove_cvref_t<decltype(declval<Token&>().try_associate())>;

```

```

    alloc-t alloc;                                // exposition
        only
    ssource-t ssource;                            // exposition
        only
    op-t op;                                       // exposition
        only
    Tokenassoc-t tokenassoc;                    // exposition
        only
        bool associated;
        // exposition only

    void destroy() noexcept;                       // exposition
        only
};
}

```

...

```
void destroy() noexcept;
```

12 *Effects*: Equivalent to:

```

auto token = std::move(this->token);
bool associated = this->associated;
auto assoc = std::move(this->assoc);
{
    auto alloc = std::move(this->alloc);

    allocator_traits<alloc-t>::destroy(alloc, this);
    allocator_traits<alloc-t>::deallocate(alloc, this, 1);
}

if (associated)
token.disassociate();

```

§ 4.4 execution::spawn

To the subsection 33.9.13.3 [exec.spawn], make the following changes:

5 Let spawn-state be the exposition-only class template:

```

namespace std::execution {
    template <class Alloc, scope_token Token, sender Sender>
    struct spawn-state : spawn-state-base {          // exposition only
        using op-t = connect_result_t<Sender, spawn-receiver>; // exposition only

        spawn-state(Alloc alloc, Sender&& sndr, Token token); // exposition only
        void complete() noexcept override;                // exposition only
        void run() noexcept;                               // exposition only

    private:
        using alloc-t =                                // exposition only
            typename allocator_traits<Alloc>::template rebind_alloc<spawn-state>;
        using assoc-t =                                     // exposition only
            remove_cvref_t<decltype(declval<Token&>().try_associate())>;

        alloc-t alloc;                                   // exposition only
        op-t op;                                         // exposition only
        Tokenassoc-t tokenassoc;                        // exposition only

        void destroy() noexcept;                        // exposition only
    };
}

```

```
spawn-state(Alloc alloc, Sender&& sndr, Token token);
```

- 6 *Effects:* ~~Initializes alloc with alloc, token with token, and op with:~~
~~— connect(std::move(sndr), spawn_receiver(this))~~ Initializes alloc with std::move(alloc), op with
connect(std::move(sndr), spawn_receiver(this)), and assoc with token.try_associate().

```
void run() noexcept;
```

- 7 *Effects:* Equivalent to:

```
if (token.try_associate()assoc)  
    start(op);  
else  
    destroycomplete();
```

```
void complete() noexcept override;
```

- 8 *Effects:* Equivalent to:

```
auto token = std::move(this->token);  
  
destroy();  
  
token.disassociate();
```

```
void destroy() noexcept;
```

- 9 *Effects:* ~~Equivalent to:~~

```
auto assoc = std::move(this->assoc);  
auto alloc = std::move(this->alloc);  
  
allocator_traits<alloc-t>::destroy(alloc, this);  
allocator_traits<alloc-t>::deallocate(alloc, this, 1);
```

§ 4.5 Async scope concepts

At the beginning of subsection 33.14.1 [\[exec.scope.concepts\]](#), make the following changes

- 1 The scope association concept defines the requirements on a type Assoc. An object of type Assoc is *engaged* if and only if it owns an association with an async scope, referred to as its *associated scope*.

```
namespace std::execution {  
    template <class Assoc>  
        concept scope_association =  
            movable<Assoc> &&  
            is_nothrow_move_constructible v<Assoc> &&  
            is_nothrow_move_assignable v<Assoc> &&  
            default_initializable<Assoc> &&  
            requires(const Assoc assoc) {  
                { static_cast<bool>(assoc) } noexcept;  
                { assoc.try_associate(). } -> same_as<Assoc>;  
            };  
}
```

- 2 A type Assoc models scope_association only if:

- (2.1) — a default constructed object of type Assoc is not engaged;
- (2.2) — for an object assoc of type Assoc, static_cast<bool>(assoc) is true if and only if assoc is engaged;
- (2.3) — no two distinct objects of type Assoc own the same association;
- (2.4) — for an object assoc of type Assoc, destroying assoc releases the association owned by assoc, if any;
- (2.5) — for an object assoc of type Assoc, after it is used as the source operand of a move constructor, the assoc is not engaged;

- (2.6) — for distinct objects `assoc1` and `assoc2` of type `Assoc`, after evaluating `assoc1 = std::move(assoc2)`, the association owned by `assoc1`, if any, is released and `assoc2` is not engaged;
 - (2.7) — for an object `assoc` of type `Assoc` that is engaged, `assoc.try_associate()` either returns an object that is not engaged or acquires a new association with `assoc`'s associated scope and returns an engaged object that owns that association;
 - (2.8) — for an object `assoc` of type `Assoc` that is not engaged, `assoc.try_associate()` returns an object that is not engaged.
- 3 The `scope_token` concept defines the requirements on a type `Token` that can be used to create associations between senders and an async scope. Every object of type `Token` is associated with an async scope that is referred to as its *associated scope*.
- 4 Let `test-sender` and `test-env` be unspecified types such that `sender_in<test-sender, test-env>` is modeled.

```
namespace std::execution {
    template <class Token>
    concept scope_token =
        copyable<Token> &&
        requires(const Token token) {
            { token.try_associate() } -> same_as;
            { token.disassociate() } noexcept -> same_as;
            { token.try_associate() } -> scope_association;
            { token.wrap(decval<test-sender>()) } -> sender_in<test-env>;
        };
}
```

- 5 A type `Token` models `scope_token` only if:
- (5.1) — no exceptions are thrown from copy construction, move construction, copy assignment, or move assignment of objects of type `Token`; ~~and~~
 - (5.2) — for an object `token` of type `Token`, `token.try_associate()` either returns an object that is not engaged or acquires a new association with `token`'s associated scope and returns an engaged object that owns that association; and
 - (5.3) — given an lvalue `token` of type...

§ 4.6 `execution::simple_counting_scope` and `execution::counting_scope`

Add the following to the end of paragraph three of 33.14.2.1 [exec.counting.scopes.general]:

```
template <class Scope>
struct association-t:
```

Add the following as paragraph five of 33.14.2.1 [exec.counting.scopes.general]:

- 5 `association-t` is a class template, specializations of which model `scope_association` and contain an exposition-only member `scope` of type `Scope*`. For a class type `Scope` and an object `assoc` of type `association-t<Scope>`:
- (5.1) — `assoc.scope` points to its associated scope,
 - (5.2) — `assoc` is engaged when `assoc.scope != nullptr` is true,
 - (5.3) — if `assoc` is engaged then `assoc.try_associate()` is equivalent to `assoc.scope->try_associate()`, and
 - (5.4) — the association owned by `assoc` is released by invoking `assoc.scope->disassociate()`.

To the subsection 33.14.2.2.1 [exec.scope.simple.counting.general], make the following change:

```
namespace std::execution {
    class simple_counting_scope {
```

```

public:
    // [exec.simple.counting.token], token
    struct token;

    // [exec.simple.counting.assoc], assoc
    using assoc-t = association-t<simple_counting_scope>; // exposition only

    static constexpr size_t max_associations = implementation-defined;

    // [exec.simple.counting.ctor], constructor and destructor
    simple_counting_scope() noexcept;
    simple_counting_scope(simple_counting_scope&&) = delete;
    ~simple_counting_scope();

    // [exec.simple.counting.mem], members
    token get_token() noexcept;
    void close() noexcept;
    sender auto join() noexcept;

private:
    size_t count; // exposition only
    scope-state-type state; // exposition only

    boolassoc-t try-associate() noexcept; // exposition only
    void disassociate() noexcept; // exposition only
    template <class State>
        bool start-join-sender(State& state) noexcept; // exposition only
};
}

```

To the subsection 33.14.2.2.3 [exec.simple.counting.mem], make the following changes:

~~bool~~assoc-t try-associate() noexcept;

5 *Effects*: If *count* is equal to *max_associations*, then no effects. Otherwise, if *state* is

- (5.1) — *unused*, then increments *count* and changes *state* to *open*;
- (5.2) — *open* or *open-and-joining*, then increments *count*;
- (5.3) — otherwise, no effects.

6 *Returns*: ~~true if count was incremented, false otherwise.~~ If *count* was incremented, an object of type *assoc-t* that is engaged and associated with **this*, and *assoc-t()* otherwise.

To the subsection 33.14.2.2.4 [exec.simple.counting.token], make the following changes:

```

namespace std::execution {
    struct simple_counting_scope::token {
        template <sender Sender>
            Sender&& wrap(Sender&& snd) const noexcept;
        boolassoc-t try_associate() const noexcept;

        void disassociate() const noexcept;

    private:
        simple_counting_scope* scope; // exposition only
    };
}

```

```

template <sender Sender>
    Sender&& wrap(Sender&& snd) const noexcept;

```

1 *Returns*: `std::forward<Sender>(snd)`.

~~bool~~assoc-t try_associate() const noexcept;

2 *Effects:* Equivalent to: `return scope->try-associate();`

~~`void disassociate() const noexcept;`~~

3 ~~*Effects:* Equivalent to `scope->disassociate();`~~

To the subsection 33.14.2.3 [exec.scope.counting], make the following changes:

```
namespace std::execution {
    class counting_scope {
    public:
        using assoc-t = association-t<counting_scope>;           // exposition only

        struct token {
            template <sender Sender>
            sender auto wrap(Sender&& snd) const noexcept(see below);
            boolassoc-t try_associate() const noexcept;

            void disassociate() const noexcept;

        private:
            counting_scope* scope;                               // exposition only
        };

        static constexpr size_t max_associations = implementation-defined;

        counting_scope() noexcept;
        counting_scope(counting_scope&&) = delete;
        ~counting_scope();

        token get_token() noexcept;
        void close() noexcept;
        sender auto join() noexcept;
        void request_stop() noexcept;

    private:
        size_t count;                                           // exposition only
        scope-state-type state;                                 // exposition only
        inplace_stop_source s_source;                           // exposition only

        boolassoc-t try_associate() noexcept;                   // exposition only
        void disassociate() noexcept;                           // exposition only

        template <class State>
        bool start-join-sender(State& state) noexcept;         // exposition only
    };
}
```

`assoc-t try-associate() noexcept;`

5 *Effects:* If `count` is equal to `max_associations`, then no effects. Otherwise, if `state` is

(5.1) — `unused`, then increments `count` and changes `state` to `open`;

(5.2) — `open` or `open-and-joining`, then increments `count`;

(5.3) — `otherwise`, no effects.

6 *Returns:* If `count` was incremented, an object of type `assoc-t` that is engaged and associated with `*this`, and `assoc-t()` otherwise.

...

`assoc-t counting_scope::token::try_associate() const noexcept;`

8 *Effects:* Equivalent to: `return scope->try-associate();`

§ 5 References

[P3149R11] Ian Petersen, Jessica Wong; Dietmar Kühl; Ján Ondrušek; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu; Ruslan Arutyunyan; 2025-06-19. `async_scope` — Creating scopes for non-sequential concurrency.

<https://wg21.link/p3149r11>

[P3149R7] Ian Petersen, Jessica Wong; Dietmar Kühl; Ján Ondrušek; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu; Ruslan Arutyunyan; 2024-11-18. `async_scope` — Creating scopes for non-sequential concurrency.

<https://wg21.link/p3149r7>

[P3149R9] Ian Petersen, Jessica Wong; Dietmar Kühl; Ján Ondrušek; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu; Ruslan Arutyunyan; 2025-02-18. `async_scope` — Creating scopes for non-sequential concurrency.

<https://wg21.link/p3149r9>